

Backend

File Path-Event Snap/Backend/server.py

File Name-Server.py

Description

This file implements backend authentication and secure access control using bcrypt and JWT. It handles password hashing, user verification, and token validation to ensure only authorized users access protected resources.

Important Functions

(a) hash_password()

```
def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')
```

(b) verify_password()

```
def verify_password(plain_password: str, hashed_password: str) -> bool:
    return bcrypt.checkpw(plain_password.encode('utf-8'), hashed_password.encode('utf-8'))
```

(c) create_access_token()

```
def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(days=7)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, JWT_SECRET, algorithm=JWT_ALGORITHM)
```

(d) get_current_user()

```
async def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security)):
```

5. Overall Functionality

This file manages authentication and authorization by securing user data, generating tokens, and validating user identity, forming the backbone of backend security.

File Path-Event Snap/Backend/face_processing.py

File Path-Event Snap/Backend/server.py

File Name- Server.py

Description This file performs face detection, embedding generation, and similarity search using FAISS for fast photo retrieval.

Important Functions

```
def generate_thumbnail(image_path: str, thumbnail_path: str, size=(300, 300)):
```

```
    try:
```

```
        img = Image.open(image_path); img.thumbnail(size, Image.Resampling.LANCZOS)
```

```
        img.save(thumbnail_path, "JPEG", quality=85); return True
```

```
    except Exception as e:
```

```
        logging.error(f"Error generating thumbnail: {e}"); return False
```

```
async def process_photo_faces(photo_id: str, event_id: str, image_path: str):
```

```
    faces = DeepFace.extract_faces(img_path=image_path, detector_backend=FACE_DETECTOR,
    enforce_detection=False)
```

```
async def build_faiss_index(event_id: str):
```

```
    embeddings_cursor = db.face_embeddings.find({"event_id": event_id}, {"_id": 0})
```

```
async def search_faces(event_id: str, query_embedding: np.ndarray, k: int = None):
```

```
    distances, indices = index.search(query_embedding.reshape(1, -1), k)
```

File Path-Event Snap/Backend/server.py

File Name-server.py

Description

This function detects faces in an image and prepares embeddings for face recognition processing.

Important Functions

```
async def process_photo_faces(photo_id: str, event_id: str, image_path: str):
```

```
    try:
```

```
        faces = DeepFace.extract_faces(
            img_path=image_path,
            detector_backend=FACE_DETECTOR,
            enforce_detection=False
        )
```

```
    if not faces:
```

```
        logging.info(f"No faces detected in photo {photo_id}")
```

```
        return []
```

```
    logging.info(f"Photo {photo_id}: detected {len(faces)} face(s)")
```

```
    face_embeddings = []
```

File Path-Event Snap/Backend/Server.py

File Name-Server.py

Description This function processes faces, generates embeddings, and stores them for recognition.

Important Functions

```
for i,f in enumerate(faces):

    if f.get('confidence',0)<0.4: continue

    r=f['facial_area']; fc=f.get('face')

    if fc is None:

        img=cv2.imread(image_path);

        if img is None: continue

        x,y,w,h=r['x'],r['y'],r['w'],r['h']; fc=img[max(0,y):y+h,max(0,x):x+w]; fs=fc

    else:

        if fc.max()<=1.0: fc=(np.clip(fc,0,1)*255).astype(np.uint8)

        fs=cv2.cvtColor(fc,cv2.COLOR_RGB2BGR) if len(fc.shape)==3 else fc

    p=STORAGE_DIR/f"f_{uuid.uuid4()}.jpg"

    try:

        cv2.imwrite(str(p),fs);
    er=DeepFace.represent(img_path=str(p),model_name=FACE_MODEL,detector_backend='skip',enforce_detection=False)

    finally:

        if p.exists(): p.unlink()

    if er:

        e=np.array(er[0]['embedding']).astype('float32'); n=np.linalg.norm(e)

        if n>0: e=e/n

        fid=str(uuid.uuid4()); b=[r['x'],r['y'],r['w'],r['h']]

        await
        db.face_embeddings.insert_one({'id':fid,'photo_id':photo_id,'event_id':event_id,'face_id':fid,'bbox':b,'embedding':e.tolist()},'created_at':datetime.now(timezone.utc).isoformat())

        face_embeddings.append((fid,e))

    return face_embeddings

except Exception as e:

    logging.error(f"Error: {e}"); return []
```

File Path-Event Snap/Backend/Server.py

File Name-Server.py

Description This function builds a FAISS index for face embeddings and performs similarity-based face search to retrieve matching photos.

Important Functions

```
async def build_faiss_index(event_id:str):
    try:
        cur=db.face_embeddings.find({"event_id":event_id},{"_id":0}); lst=await cur.to_list(None)
        if not lst: return
        fids=[]; embs=[]; pmap={}
        for d in lst:
            fids.append(d['face_id']); e=np.array(d['embedding']).astype('float32')
            n=np.linalg.norm(e); e=e/n if n>0 else e
            embs.append(e); pmap[d['face_id']]=d['photo_id']
        arr=np.array(embs).astype('float32')
        idx=faiss.IndexFlatL2(arr.shape[1]); idx.add(arr)
        faiss_indices[event_id]=idx; face_mappings[event_id]={'face_ids':fids,'photo_map':pmap}
        ip=FAISS_DIR/f"{event_id}.index"; mp=FAISS_DIR/f"{event_id}.pkl"
        faiss.write_index(idx,str(ip)); pickle.dump({'face_ids':fids,'photo_map':pmap},open(mp,'wb'))
    except Exception as e:
        logging.error(f"Error: {e}")

async def search_faces(event_id:str,q:np.ndarray,k:int=None):
    try:
        if event_id not in faiss_indices:
            ip=FAISS_DIR/f"{event_id}.index"; mp=FAISS_DIR/f"{event_id}.pkl"
            if not ip.exists(): return []
            faiss_indices[event_id]=faiss.read_index(str(ip)); face_mappings[event_id]=pickle.load(open(mp,'rb'))
        idx=faiss_indices[event_id]; m=face_mappings[event_id]; n=idx.ntotal
        if n==0: return []
        k=n if (k is None or k>n) else k
        d,i=idx.search(q.reshape(1,-1).astype('float32'),k); ps={}
        for x,dist in zip(i[0],d[0]):
```

```
if x==-1: continue

fid=m['face_ids'][x]; pid=m['photo_map'][fid]; s=float(1-(dist*dist/2))

if pid not in ps or s>ps[pid]: ps[pid]=s

res=[]

for pid,bs in ps.items():

    if bs<MINIMUM_MATCH_THRESHOLD: continue

    p=await db.photos.find_one({"id":pid},{ "_id":0})

    if p:
res.append(SearchResult(photo_id=pid,event_id=event_id,similarity_score=bs,thumbnail_url=f"/api/photos/{pid}/thumbnail",p
hoto_url=f"/api/photos/{pid}"))

    res.sort(key=lambda x:x.similarity_score,reverse=True); return res

except Exception as e:

    logging.error(f"Error: {e}"); return []
```

File Path-Event Snap/Backend/Server.py

File Name-Server.py

Description This code handles user registration, login, and authentication by generating and validating JWT tokens.

Important Functions

```
@api_router.post("/auth/register")
```

```
async def register(user:UserCreate):
```

```
    u=await db.users.find_one({"email":user.email})
```

```
    if u: raise HTTPException(status_code=400,detail="Email already registered")
```

```
    uid=str(uuid.uuid4())
```

```
    doc={'id':uid,'email':user.email,'password_hash':hash_password(user.password),'name':user.name,'role':user.role,'created_at':datetime.now(timezone.utc).isoformat()}
```

```
    await db.users.insert_one(doc)
```

```
    t=create_access_token({"sub":uid})
```

```
    return {'token':t,'user':{'id':uid,'email':user.email,'name':user.name,'role':user.role}}
```

```
@api_router.post("/auth/login")
```

```
async def login(c:UserLogin):
```

```
    u=await db.users.find_one({"email":c.email})
```

```
    if not u or not verify_password(c.password,u['password_hash']):
```

```
        raise HTTPException(status_code=401,detail="Invalid credentials")
```

```
    t=create_access_token({"sub":u['id']})
```

```
    return {'token':t,'user':{'id':u['id'],'email':u['email'],'name':u['name'],'role':u['role']}}
```

```
@api_router.get("/auth/me")
```

```
async def get_me(current_user:User=Depends(get_current_user)):
```

```
    return current_user
```

File Path-Event Snap/Backend/Server.py

File Name-Server.py

Description This function allows organizers to create events by storing event details and generating a unique event code.

Important Functions

```
@api_router.post("/events")
```

```
async def create_event(event:EventCreate,current_user:User=Depends(get_current_user)):
```

```
    if current_user.role!="organizer":
```

```
        raise HTTPException(status_code=403,detail="Only organizers can create events")
```

```
    eid=str(uuid.uuid4()); code=secrets.token_urlsafe(8).upper()[:8]
```

```
    doc={'id':eid,'organizer_id':current_user.id,'title':event.title,'description':event.description,'event_code':code,'date':event.date,'status':'active','total_photos':0,'processed_photos':0,'faces_detected':0,'created_at':datetime.now(timezone.utc).isoformat()}
```

```
    await db.events.insert_one(doc)
```

```
    doc['created_at']=datetime.fromisoformat(doc['created_at'])
```

```
    return Event(**doc)
```


File Path-Event Snap/Backend/Server.py

File Name-Server.py

Description This function uploads event photos, stores them, generates thumbnails, and triggers background processing.

Important Functions

```
@api_router.post("/events/{event_id}/upload")

async def
upload_photos(event_id:str,files:List[UploadFile]=File(...),current_user:User=Depends(get_curre
nt_user)):

    e=await db.events.find_one({"id":event_id})

    if not e: raise HTTPException(status_code=404,detail="Event not found")

    if e['organizer_id']!=current_user.id: raise HTTPException(status_code=403,detail="Not
authorized")

    up=[]

    for file in files:

        pid=str(uuid.uuid4()); ext=file.filename.split('.')[-1]

        pf=PHOTOS_DIR/f"{pid}.{ext}"

        async with aiofiles.open(pf,'wb') as f: await f.write(await file.read())

        tf=THUMBNAILS_DIR/f"{pid}_thumb.jpg"; generate_thumbnail(str(pf),str(tf))

        await
        db.photos.insert_one({'id':pid,'event_id':event_id,'file_path':str(pf),'thumbnail_path':str(tf),'upl
oad_status':'uploaded','processed':False,'faces_detected':0,'created_at':datetime.now(timezon
e.utc).isoformat()})

        up.append(pid)

    await db.events.update_one({"id":event_id},{"$inc":{"total_photos":len(files)}})

    asyncio.create_task(process_event_photos(event_id))

    return {'message':f'{len(files)} photos uploaded successfully','photo_ids':up}
```

File Path-Event Snap/Backend/Server.py

File Name-Server.py

Description This function processes uploaded event photos, detects faces, updates database records, and builds the FAISS index.

Important Functions

```
async def process_event_photos(event_id:str):
    try:
        await db.events.update_one({"id":event_id},{"$set":{"status":"processing"}})
        photos=await db.photos.find({"event_id":event_id,"processed":False},{"_id":0}).to_list(None)
        for p in photos:
            try:
                fe=await process_photo_faces(p['id'],event_id,p['file_path'])
                await
                db.photos.update_one({"id":p['id']},{"$set":{"processed":True,"faces_detected":len(fe),"upload_status":"completed"}})
                await
                db.events.update_one({"id":event_id},{"$inc":{"processed_photos":1,"faces_detected":len(fe)}}
            )
            except Exception as e:
                logging.error(f"Error photo {p['id']}: {e}")
                await db.photos.update_one({"id":p['id']},{"$set":{"upload_status":"failed"}})
                await build_faiss_index(event_id)
                await db.events.update_one({"id":event_id},{"$set":{"status":"completed"}})
        except Exception as e:
            logging.error(f"Error event: {e}")
            await db.events.update_one({"id":event_id},{"$set":{"status":"failed"}})
```

File Path-Event Snap/Backend/Server.py

File Name-Server.py

Description This code retrieves original photos and thumbnails from storage based on photo ID.

Important Functions

```
@api_router.get("/photos/{photo_id}")
```

```
async def get_photo(photo_id:str):
```

```
    p=await db.photos.find_one({"id":photo_id},{"_id":0})
```

```
    if not p: raise HTTPException(status_code=404,detail="Photo not found")
```

```
    return FileResponse(p['file_path'])
```

```
@api_router.get("/photos/{photo_id}/thumbnail")
```

```
async def get_photo_thumbnail(photo_id:str):
```

```
    p=await db.photos.find_one({"id":photo_id},{"_id":0})
```

```
    if not p: raise HTTPException(status_code=404,detail="Photo not found")
```

```
    return FileResponse(p['thumbnail_path'])
```

Frontend

File Path-Event Snap/Frontend/src/pages/AttendeeEntry.js

File Name-AttendeeEntry.js

Description This component allows users to enter an event code, validate it, and navigate to the photo search page.

Important Functions

```
const handleFind=async(e)=>{  
  e.preventDefault(); if(!code.trim()) return; setLoading(true);  
  
  try{  
    const r=await api.get(`/events/code/${code.toUpperCase().trim()}`); const ev=r.data;  
    if(ev.status==='processing'){toast.error('Event processing'); setLoading(false); return;}  
    if(ev.status==='active'&&ev.total_photos===0){toast.error('No photos'); setLoading(false);  
return;}  
    if(ev.faces_detected===0) toast.warning('No faces detected');  
    toast.success(`Found: ${ev.title}`); navigate(`/attend/${ev.id}/selfie`,{state:{event:ev}});  
  }catch(err){toast.error(err.response?.data?.detail || 'Event not found');}  
  finally{setLoading(false);}  
}
```

File Path-Event Snap/Frontend/src/pages/Auth.js

File Name-Auth.js

Description This component handles user login and signup, manages authentication state, and redirects users based on their role.

Important Functions

```
const handleLogin=async(e)=>{  
  e.preventDefault(); setLoading(true);  
  try{  
    const r=await api.post('/auth/login',loginData);  
    localStorage.setItem('token',r.data.token);  
    localStorage.setItem('user',JSON.stringify(r.data.user));  
    navigate(r.data.user.role==='organizer'?'/dashboard': '/attend');  
  }catch(err){toast.error(err.response?.data?.detail || 'Login failed');}  
  finally{setLoading(false);}  
}  
  
const handleSignup=async(e)=>{  
  e.preventDefault(); setLoading(true);  
  try{  
    const r=await api.post('/auth/register',signupData);  
    localStorage.setItem('token',r.data.token);  
    localStorage.setItem('user',JSON.stringify(r.data.user));  
    navigate(r.data.user.role==='organizer'?'/dashboard': '/attend');  
  }catch(err){toast.error(err.response?.data?.detail || 'Signup failed');}  
  finally{setLoading(false);}  
}
```

File Path-Event Snap/Frontend/src/pages/Landing.js

File Name-Landing.js

Description This component renders the landing page by combining sections like Hero, Features, and call-to-action for user navigation.

Important Functions

```
export default function Landing(){
  return(
    <div className="bg-slate-950">
      <Hero/><HowItWorks/><Features/>
      <section className="relative py-28 overflow-hidden">
        <div className="absolute inset-0 bg-gradient-to-r from-indigo-600 via-violet-600 to-purple-700"/>
        <div className="relative z-10 max-w-4xl mx-auto px-6 text-center">
          <motion.div initial={{opacity:0,y:20}} whileInView={{opacity:1,y:0}}>
            <h2 className="text-4xl font-bold text-white mb-4">Ready to transform your event
photos?</h2>
            <p className="text-indigo-200 mb-10">Join organizers using AI face recognition</p>
            <div className="flex gap-4 justify-center">
              <a href="/auth" className="bg-white text-indigo-700 px-8 py-4 rounded-xl">Get Started
→</a>
              <a href="/attendjoin" className="border text-white px-8 py-4 rounded-xl">Find Photos</a>
            </div>
          </motion.div>
        </div>
      </section>
      <LandingFooter/>
    </div>)}
}
```

File Path-Event Snap/Frontend/src/pages/PhotoGallery.js

File Name-PhotoGallery.js

Description This component displays matched photos, allows selection, and supports single and bulk download of images.

Important Functions

```
const toggleSelect=id=>{  
  const s=new Set(selectedPhotos); s.has(id)?s.delete(id):s.add(id); setSelectedPhotos(s);  
}
```

```
const download=p=>{  
  const url=`${BACKEND_URL}${p.photo_url}`; window.open(url,'_blank');  
}
```

```
const bulkDownload=()=>{  
  if(selectedPhotos.size===0) return toast.error("Select photos first");  
  selectedPhotos.forEach(id=>{  
    const p=results.find(r=>r.photo_id===id);  
    if(p) window.open(`${BACKEND_URL}${p.photo_url}`);  
  });  
  toast.success(`Downloading ${selectedPhotos.size} photos`);  
}
```

File Path-Event Snap/Frontend/src/pages/SelfieCapture.js

File Name-SelfieCapture.js

Description This component captures or uploads a selfie, sends it to the backend, and retrieves matching photos using face recognition.

Important Functions

```
const capture=()=>{setImgSrc(webcamRef.current.getScreenshot())}
```

```
const handleFileUpload=e=>{
```

```
  const f=e.target.files[0]; if(!f || !f.type.match('image.*')) return toast.error('Invalid file')
```

```
  const r=new FileReader();
```

```
  r.onload=ev=>{setImgSrc(ev.target.result);setUploadedFile(f);setUploadMode(true)};
```

```
  r.readAsDataURL(f)
```

```
}
```

```
const handleSearch=async()=>{
```

```
  if(!imgSrc) return; setSearching(true)
```

```
  try{
```

```
    let b=uploadedFile || await(await fetch(imgSrc)).blob()
```

```
    const fd=new FormData(); fd.append('file',b,'selfie.jpg'); fd.append('event_id',eventId)
```

```
    const res=await api.post('/search/selfie',fd,{headers: {'Content-Type':'multipart/form-data'}})
```

```
    if(res.data.length===0){toast.error('No photos found'); setImgSrc(null)}
```

```
    else{toast.success(`Found ${res.data.length}`);
```

```
    navigate(`/attend/${eventId}/gallery`,{state:{results:res.data,event}})}
```

```
  }catch(err){toast.error(err.response?.data?.detail || 'Search failed'); setImgSrc(null)}
```

```
  finally{setSearching(false)}
```

```
}
```


File Path-Event Snap/Frontend/src/pages/EventDetails.js

File Name-EventDetails.js

Description This component loads event details, tracks processing status, and manages event photos including upload and display.

Important Functions

```
useEffect(()=>{  
  loadEventDetails(); loadPhotos();  
  const i=setInterval(loadStatus,3000); return()=>clearInterval(i)  
},[eventId])
```

```
const loadEventDetails=async()=>{  
  try{  
    const r=await api.get(`/events/${eventId}`);  
    setEvent(r.data); await loadStatus()  
  }catch{toast.error('Failed to load event')}  
  finally{setLoading(false)}  
}
```

```
const loadPhotos=async()=>{  
  try{const r=await api.get(`/events/${eventId}/photos`); setPhotos(r.data)}  
  catch{console.error('Failed photos')}  
}
```

```
const loadStatus=async()=>{  
  try{const r=await api.get(`/events/${eventId}/status`); setStatus(r.data)}  
  catch{console.error('Failed status')}  
}
```

File Path-Event Snap/Frontend/src/pages/EventDetails.js

File Name-EventDetails.js

Description This code handles photo deletion, uploading with progress tracking, and reprocessing of event data.

Important Functions

```
const handleDeletePhoto=async()=>>{  
  if(!photoToDelete) return; setDeleting(true)  
  
  try{  
    await api.delete(`/events/${eventId}/photos/${photoToDelete.id}`)  
    toast.success('Deleted'); setDeleteDialogOpen(false); setPhotoToDelete(null)  
    await loadPhotos(); await loadEventDetails()  
  }catch(e){toast.error(e.response?.data?.detail || 'Delete failed')}  
  finally{setDeleting(false)}}  
  
const handleFileUpload=async(e)=>{  
  const f=Array.from(e.target.files); if(!f.length) return  
  setUploading(true); setUploadProgress(0)  
  
  try{  
    const fd=new FormData(); f.forEach(x=>fd.append('files',x))  
    await api.post(`/events/${eventId}/upload`,fd,{headers:{'Content-Type':'multipart/form-data'},  
      onUploadProgress:p=>setUploadProgress(Math.round((p.loaded*100)/p.total))})  
    toast.success(`${f.length} uploaded`); await loadPhotos(); setTimeout(loadEventDetails,1000)  
  }catch(e){toast.error(e.response?.data?.detail || 'Upload failed')}  
  finally{setUploading(false); setUploadProgress(0)}}  
  
const handleReprocess=async()=>>{  
  setReprocessing(true)  
  
  try{await api.post(`/events/${eventId}/reprocess`)  
    toast.success('Rebuilding index'); await loadStatus(); await loadEventDetails()  
  }catch(e){toast.error(e.response?.data?.detail || 'Reprocess failed')}  
  finally{setReprocessing(false)}}
```

File Path-Event Snap/Frontend/src/pages/EventDetails.js

File Name-EventDetails.js

Description This code displays event data, handles upload UI, and shows photos.

Important Functions

```
if (loading) return <div className="min-h-screen flex items-center justify-center"><div className="text-center"><div className="animate-spin h-12 w-12 border-b-2 border-indigo-600 mx-auto mb-4"></div><p>Loading...</p></div></div>
```

```
const p = status && status.total_photos > 0 ? (status.processed_photos / status.total_photos) * 100 : 0
```

```
return <div className="min-h-screen bg-slate-50">
```

```
<header className="bg-white border-b"><div className="max-w-7xl mx-auto px-4 py-4">
```

```
<Button onClick={() => navigate('/dashboard')}><ArrowLeft/>Back</Button></div></header>
```

```
<main className="max-w-7xl mx-auto px-4 py-8">
```

```
<div className="bg-white p-8 mb-8"><h1>{event?.title}</h1><p>{event?.description}</p></div>
```

```
<div className="bg-white p-8"><input type="file" multiple onChange={handleFileUpload}/>
```

```
{uploading && <Progress value={uploadProgress}/>}</div>
```

```
<div className="grid grid-cols-3 gap-4 mt-8">
```

```
{photos.map(ph => <div key={ph.id}>
```

```
<img src={` ${BACKEND_URL}/api/photos/${ph.id}/thumbnail`}/>
```

```
<Button onClick={() => {setPhotoToDelete(ph); setDeleteDialogOpen(true)}}>Delete</Button>
```

```
</div>)}</div>
```

```
</div>
```

```
</main></div>
```

File Path-Event Snap/Frontend/src/pages/OrganizerDashboard.js

File Name-OrganizerDashboard.js

Description This code loads organizer data, fetches events, handles event creation, and manages logout.

Important Functions

```
useEffect(()=>{  
  const u=JSON.parse(localStorage.getItem('user')) || '{}'; setUser(u); loadEvents()  
},[])
```

```
const loadEvents=async()=>{  
  const r=await api.get('/events'); setEvents(r.data)  
}
```

```
const handleCreateEvent=async(e)=>{  
  e.preventDefault()  
  const r=await api.post('/events',newEvent)  
  setEvents([r.data,...events]); setCreateDialogOpen(false)  
  setNewEvent({title:"",description:"",date:""})  
}
```

```
const handleLogout=()=>{  
  localStorage.removeItem('token'); localStorage.removeItem('user'); navigate('/')  
}
```

File Path-Event Snap/Frontend/src/pages/OrganizerDashboard.js

File Name-OrganizerDashboard.js

Description This code renders the organizer dashboard UI, including event creation and listing of all events.

Important Functions

```
return(  
  <div className="min-h-screen bg-slate-50">  
    <header className="bg-white border-b">  
      <div className="flex justify-between p-4">  
        <h1>FaceShot</h1>  
        <Button onClick={handleLogout}>Logout</Button>  
      </div>  
    </header>  
    <main className="p-6">  
      <Button onClick={()=>setCreateDialogOpen(true)}>Create Event</Button>  
  
      <div className="grid grid-cols-3 gap-4 mt-6">  
        {events.map(e=>(  
          <div key={e.id} onClick={()=>navigate(`/events/${e.id}`)} className="p-4 border rounded cursor-pointer">  
            <h3>{e.title}</h3>  
            <p>Code: {e.event_code}</p>  
            <p>{e.total_photos} photos</p>  
          </div>  
        ))}  
      </div>  
    </main>  
  </div>)
```

